CyberSecurity CTF Flags

01 Old Sessions (Replacing the sessions with the ADMIN session to login)

Proper session timeout controls are critical for securing user accounts. If a user logs in on a public or shared computer but doesn't explicitly log out (instead simply closing the browser tab), and session expiration dates are misconfigured, the session may remain active indefinitely.

This then allows an attacker using the same browser later to access the user's account without needing credentials, exploiting the fact that sessions never expire and remain authenticated.

Step 1: Register the Login Page
Step 2: Login the website with the ID & Password
Step 3: Ctrl + i to open inspect, find the session key
Step 4: Find the other session keys, by pasting '/sessions' in the URL
Step 5: Copy the Admin session key, replayed it with your session key
Step 6: Refresh the URL, get logged into the Admin Account

---

02 Crack the Gate 1 (Logging without Password)

We're in the middle of an investigation. One of our persons of interest, ctf player, is believed to be hiding sensitive data inside a restricted web portal. We've uncovered the email address he uses to log in: `ctf-player@cylabacademy.org`. Unfortunately, we don't know the password, and the usual guessing techniques haven't worked. But something feels off... it's almost like the developer left a secret way in. Can you figure it out?

Step 1: Ctrl + U to inspect the source page
Step 2: Find the suspicious cyber text (unreadable text)
Step 3: Decode that by using 'ROT13'
Step 4: Bypass the Header by Ctrl + j to open console
Step 5: Allow pasting by typing 'Allow Pasting'
Step 6: Paste the javascript code

```
      javascript
fetch('/login', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json', (Found by ROT13)
        'X-Dev-Access': 'yes'(Found by ROT13)
    },
    body: JSON.stringify({
        email: 'ctf-player@cylabacademy.org',
        password: 'anything'
    })
})
.then(response => response.json())
.then(data => console.log(data));
```

Step 7: Got the Flag

---

03 SSTI1 (Os command injection Via SSTI)

SSTI1 from picoCTF 2025, is a classic Server-Side Template Injection (SSTI) problem. The key is to confirm the vulnerability by getting the server to execute a calculation, then use a payload to read the flag file.

Step 1: Confirm the SSTI Vulnerability: Enter `{{7*7}}` in the text box on the page. If the server returns `49` instead of the literal string `{{7*7}}`, it means your input is being evaluated as code by a template engine (such as Jinja2)

Step 2: Execute Commands via the `cycler` Payload: Jinja2 exposes the `cycler` object, which can be used to access Python's built-in modules and execute system commands. First, list the files in the current directory to locate the flag :

1. `jinja2`
2. `{{ cycler.__init__.__globals__.os.popen('ls -la').read() }}`

---

04 Cookie Monster Secret Recipe

The objective was to find a hidden flag (the "secret recipe") on a web application dedicated to the "Cookie Monster." The hints suggested checking the webpage, using browser tools, and looking at cookies.

Step 1: Initial Investigation & Login Attempt **(Access denied)**
Step 2: Checking Application Storage (The Dead End) Application tab and checked Local Storage, Session Storage, and Cookies.
Step 3: Console Experimentation (A Minor Detour), Console and typed document.cookie, found “ “ empty string
Step 4: Network tap in console and again typed test in both Login & Password to find out the login.php, then viewed header and found secret_recipe. There was the cypher text after secret_recipe
Step 5: Used **Base64decode** to decode the cypher text

---

05 WebDecode

Step 1: Went to about section by adding /about.html at the end of the URL
Step 2: Element section
Step 3: Under body
Step 4: Got the cyphertext (Flag)
Step 5: Decode it with cyberchef from base64 to original text

---

06 Unminify

Step 1: Source code by Ctrl+U
Step 2: scroll and find the flag